Escola de Engenharia
Programa de Pós-Graduação em Engenharia Elétrica

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

LISTA 2

Belo Horizonte
2 de abril de 2026

SUMÁRIO

1 PROBLEMA 3.4 . . . . . **3**
1.1 Solução analítica 3
1.2 Solução numérica 3
2 PROBLEMA 3.5 . . . . . **6**

ANEXO A CÓDIGO QUESTAO 3.4 . . . . . **8**

1 PROBLEMA 3.4

1.1 Solução analítica

A solução analítica vem da equação de D'Alembert, conforme descrito no livro-texto:

$$\mathcal{V}(x,t) = \frac{1}{2}\left[\phi(x - v_p t) + \phi(x + v_p t) + \frac{1}{v_p}\int_{x-v_p t}^{x+v_p t} \theta(\zeta) d\zeta\right]$$

Temos

- ϕ é a função da condição inicial de $\mathcal{V}(x,t)$, dada no enunciado
- θ é a função da condição inicial de $\mathcal{V}(x,t)$, que assumimos ser zero uma vez que a corrente inicial é nula

Assim,

$$\mathcal{V}(x,t) = \frac{1}{2}\left[\phi(x - v_p t) + \phi(x + v_p t)\right]$$

Temos duas ondas se propagando em direções opostas com metade da amplitude inicial.

1.2 Solução numérica

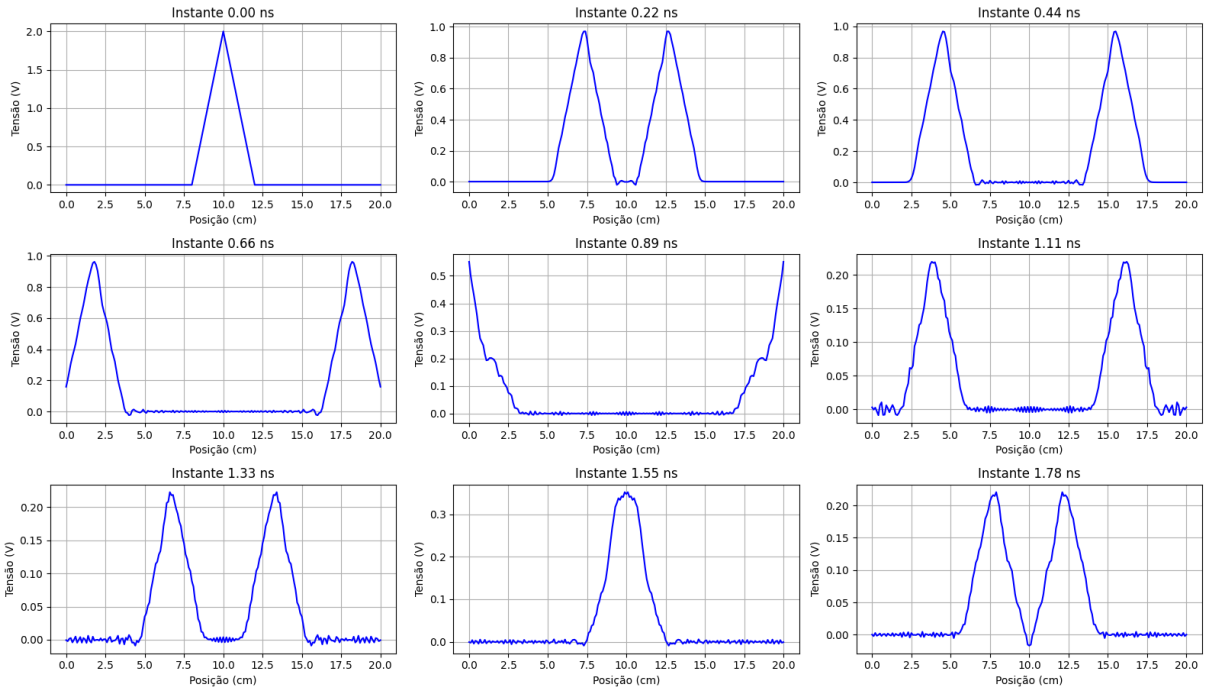
O código usado nessa solução encontra-se no Anexo A.

Os parâmetros de discretização foram selecionados da seguinte forma:

- $\Delta t = 5$ ps: isso vem do fato de que a velocidade da onda é de $v_p = 1.25 \cdot 10^{10}$ cm/s. Como o comprimento da linha é de 20 cm, o tempo de propagação é de $t_p = \frac{20 \text{ cm}}{1.25 \cdot 10^{10} \text{ cm/s}} = 1.6$ ns. Assim, um passo de tempo de 5 ps é suficientemente pequeno para capturar a dinâmica da propagação da onda.
- $\Delta x = 0.1$ cm: isso é escolhido para garantir que tenhamos uma resolução espacial adequada para capturar a forma da onda, obtendo-se 200 pontos espaciais ao longo da linha de 20 cm.

A Figura 1 apresenta a solução numérica para diferentes instantes de tempo. Ela mostra como a onda se propaga como duas ondas viajando em direções opostas, uma para a direita e outra para a esquerda, a partir da condição inicial dada, conforme previsto na solução analítica.

Figura 1 – Solução numérica da questão 3.4.



Além disso, note que a condição de contorno ”A“ usada simula uma linha de transmissão infinita, de modo que não temos reflexões no sinal nas extremidades.

Variando-se o parâmetro Δx, vemos que valores muito grandes levam a instabilidades numéricas, enquanto valores muito pequenos levam a um aumento significativo no tempo de execução do código e uma mudança na solução obtida, como mostra a Figura 2.

De modo similar, variando-se o parâmetro Δt, vemos que valores muito grandes levam a instabilidades numéricas, enquanto valores muito pequenos levam a uma melhor resolução na forma de onda, apesar de um aumento no tempo de execução do código.

Figura 2 – Variações de Δx na solução da questão 3.5.

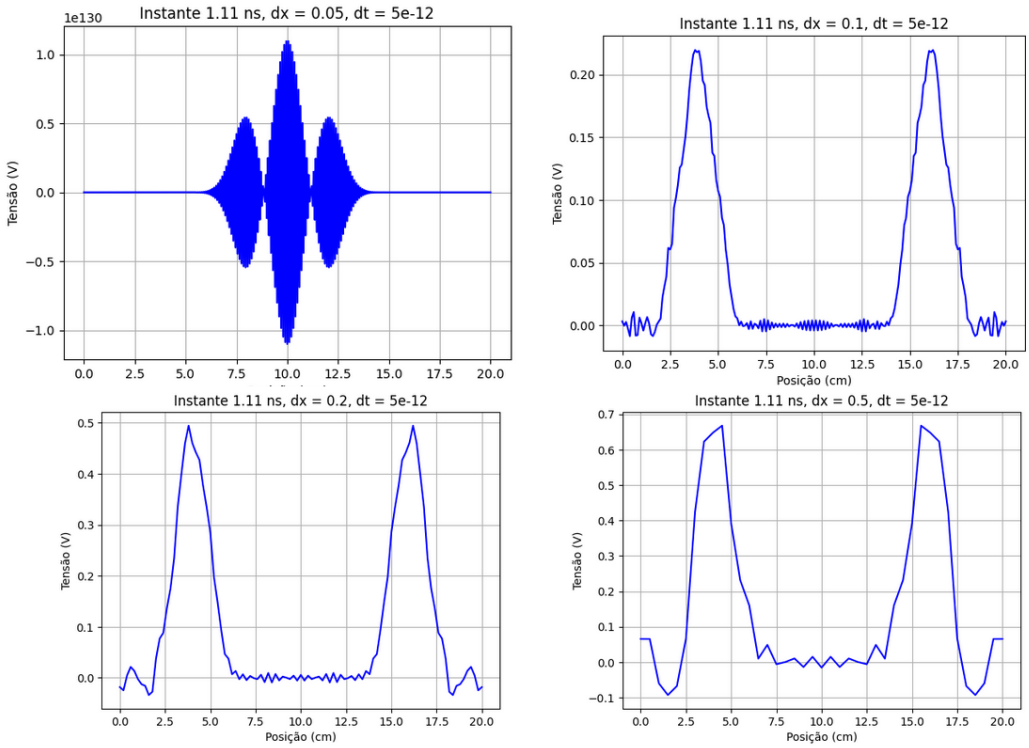
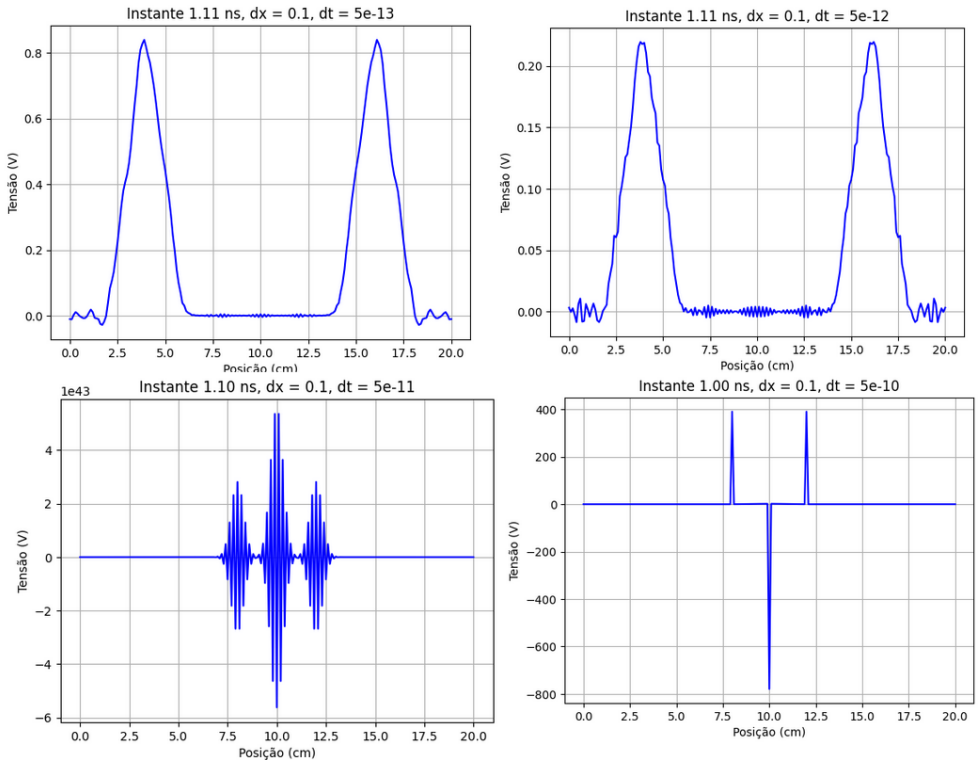


Figura 3 – Variações de Δt na solução da questão 3.5.



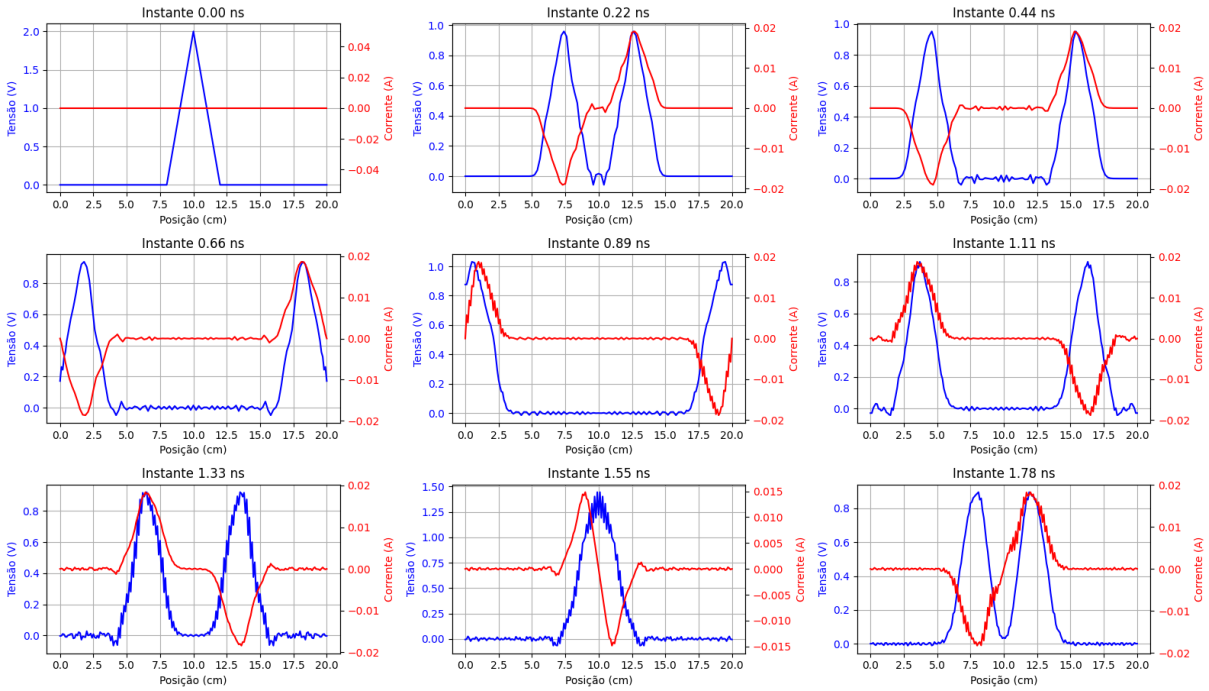
2 PROBLEMA 3.5

Para resolver ambas equações ao mesmo tempo, basta resolver elas juntas no loop principal, com a seguinte adaptação ao código da questão 3.4:

```
# Loop Principal
for n in range(1, nt - 1):
        V[n+1, 1:-1] = V[n-1, 1:-1] - coeff_V * (I[n, 2:] - I[n, :-2])
        I[n+1, 1:-1] = I[n-1, 1:-1] - coeff_I * (V[n, 2:] - V[n, :-2])
```

Usando os mesmos parâmetros de discretização da questão anterior, obtemos a seguinte solução numérica na Figura 4.

Figura 4 – Solução numérica da questão 3.5.



Variando-se os parâmetros Δx e Δt, vemos comportamentos semelhantes aos observados na questão 3.4.

Figura 5 – Variações de Δx na solução da questão 3.5.

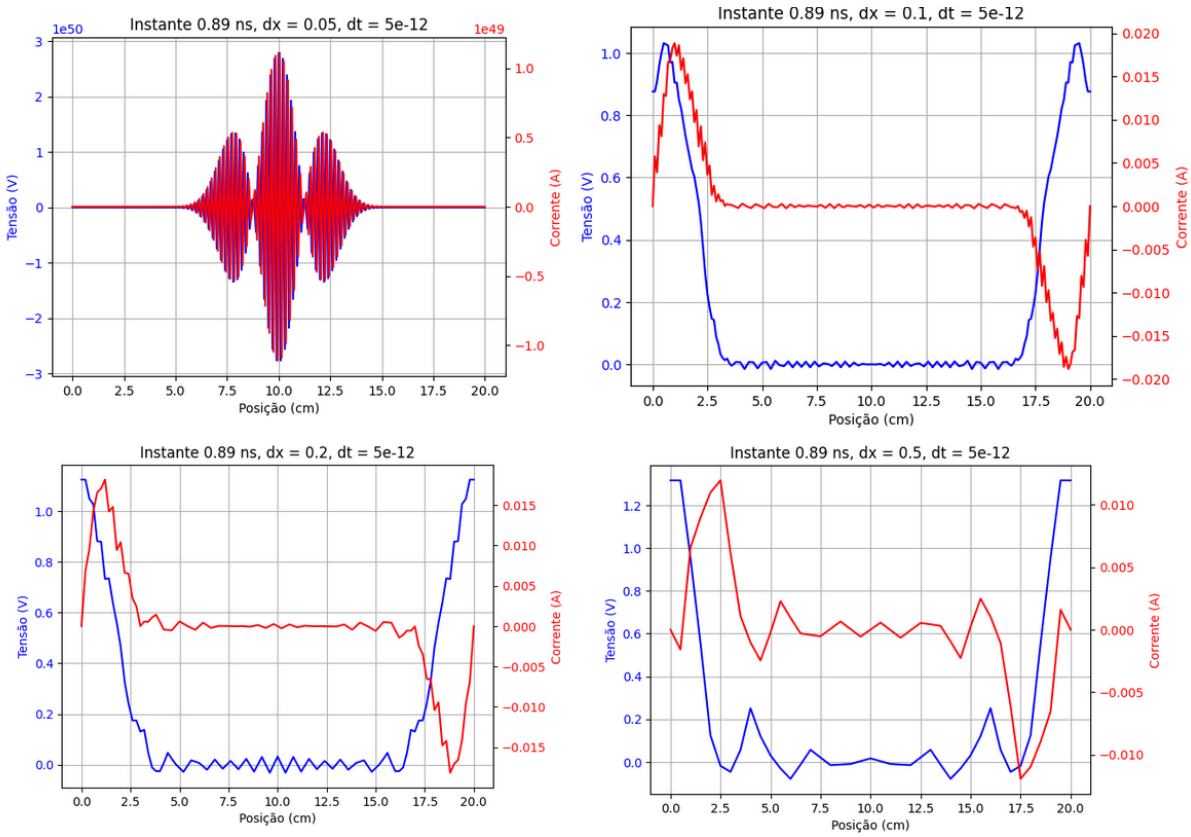
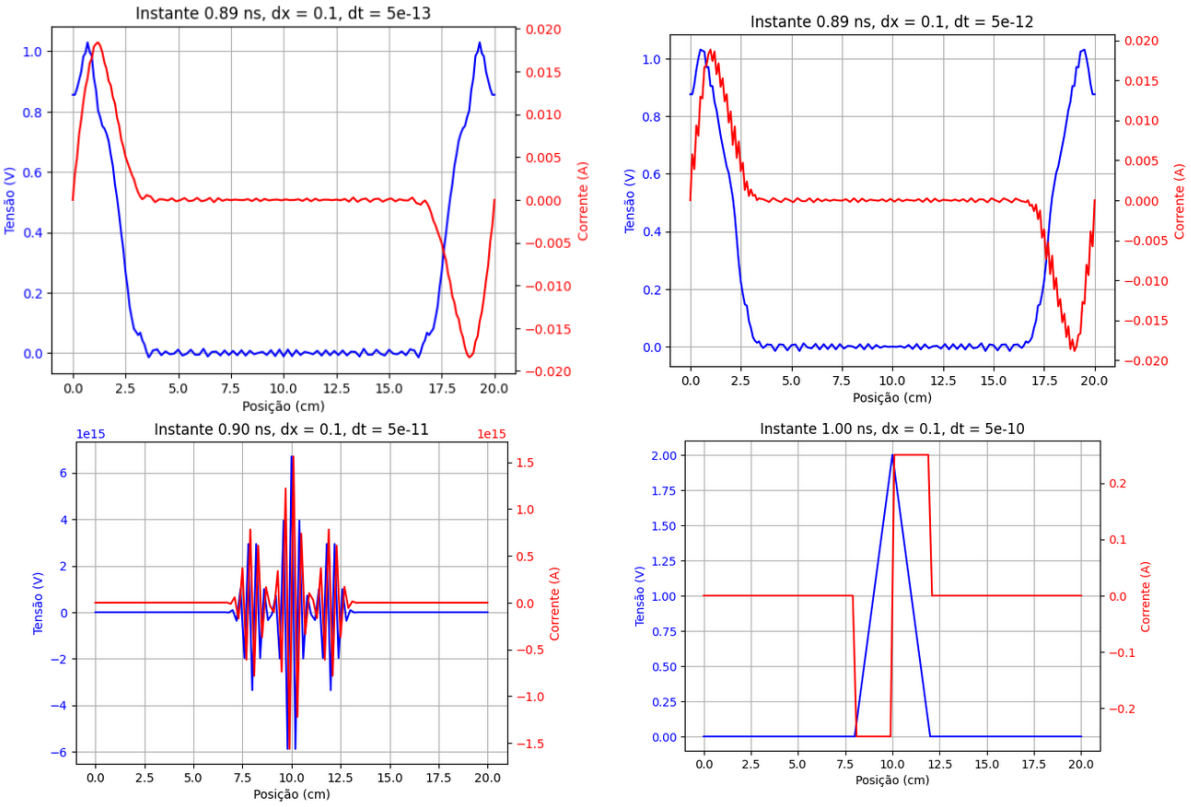


Figura 6 – Variações de Δt na solução da questão 3.5.



ANEXO A – CÓDIGO QUESTAO 3.4

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

def f(x):
    if 8 <= x <= 10: return x - 8
    elif 10 < x <= 12: return 12 - x
    else: return 0

# Parameters
dt = 5e-12
dx = 0.1
x_max = 20 # transmission line length
t_max = 2e-9 # final time
L = 4e-9
C = 1.6e-12
Z0 = np.sqrt(L / C) # characteristic impedance
vp = 1 / np.sqrt(L * C) # wave velocity

# Parametros de grade
nx = int(x_max / dx) + 1
nt = int(t_max / dt) + 1
C2 = (vp * dt / dx)**2

# Inicializacao da matriz de potencial V[tempo, espaco]
V = np.zeros((nt, nx))

# condicao inicial
x = np.linspace(0, x_max, nx)
initial = np.zeros(nx)

for i in range (nx):
    initial[i] = f(x[i])

V[0, :] = initial

# Primeiro passo de tempo
for i in range(1, nx - 1):
    V[1, i] = V[0, i]

# Loop Principal
for n in range(1, nt - 1):
    V[n+1, 1:-1] = C2 * (V[n, 2:] - 2*V[n, 1:-1] + V[n, :-2]) + 2*V[n, 1:-1] - V[n-1,
        1:-1]

    # condicoes de contorno
    V[n+1, 0] = V[n, 1] # em x = 0
    V[n+1, -1] = V[n, -2] # em x = 20
```